



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №8
Технології розробки програмного забезпечення
«ШАБЛОНИ «COMPOSITE»,
«FLYWEIGHT», «INTERPRETER»,
«VISITOR»»

Виконав:
студент групи ІА-24
Іскандаров Е. Е

Перевірив:
Мягкий М. Ю.

Тема: ШАБЛОНИ «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

Зміст

Короткі теоретичні відомості.....	2
Реалізація шаблону проектування	2
Структура шаблону	5

Короткі теоретичні відомості

Шаблон Composite використовується для складання об'єктів в деревоподібну структуру для подання ієрархій типу «частина цілого». Даний шаблон дозволяє уніфіковано обробляти як поодинокі об'єкти, так і об'єкти з вкладеністю. Даний шаблон зручно використовувати при необхідності подання та обробки ієрархій об'єктів.

Шаблон Flyweight використовується для зменшення кількості об'єктів в додатку, шляхом поділу їх між різними ділянками програми. Він дозволяє створювати поділювані об'єкти, де внутрішній стан зберігається в самому об'єкті, а зовнішній стан зберігається в контексті використання, що дозволяє ефективно управляти ресурсами при роботі з великою кількістю схожих об'єктів.

Шаблон Interpreter використовується для подання граматики мови і створення інтерпретатора, який обчислює значення виразів на основі абстрактного синтаксичного дерева. Кожен елемент граматики інтерпретується окремо, а батьківські вирази рекурсивно обчислюють дочірні елементи, що дозволяє легко розширювати граматику та додавати нові способи інтерпретації. Шаблон зручний для невеликих граматик і простих контекстів.

Шаблон Visitor дозволяє додавати нові операції для об'єктів без зміни їх структури, забезпечуючи розділення логіки операцій і даних елементів. Він зручний для групування однотипних операцій, але ускладнює додавання нових елементів в ієрархію, оскільки вимагає змін у всіх відвідувачах.

Хід роботи:

Тема:

..6 Web-browser (proxy, chain of responsibility, factory method, template method, visitor, p2p)

Веб-браузер повинен мати можливість зробити наступне: мати адресний рядок для введення адреси сайту, переміщатися і відображати структуру html документа, переглядати підключений javascript та css файли, перегляд всіх підключених ресурсів (зображень), коректна обробка відповідей з сервера (коди відповідей HTTP) - переходи при перенаправленнях, відображення сторінок 404 і 502/503.

Патерн **Visitor** дозволяє додавати нову поведінку до класів без зміни їхнього коду, що підходить для реалізації описаних можливостей веб-браузера, таких як обробка різних ресурсів (HTML, CSS, JavaScript, зображення) та обробка HTTP-кодів відповідей.

```
3  abstract class BrowserResource { 8 usages 4 inheritors
4      public abstract void accept(ResourceVisitor visitor); 4 usages 4 implementations
5  }
```

Рис. 1.1 – Код абстрактного класу BrowserResource

BrowserResource: Абстрактний клас для різних типів ресурсів (HTML, CSS, JS, Images).

```
3      class CssResource extends BrowserResource { 1 usage
4          private String cssContent; 2 usages
5
6          public CssResource(String cssContent) { no usages
7              this.cssContent = cssContent;
8          }
9
10         public String getCssContent() { no usages
11             return cssContent;
12         }
13
14         @Override no usages
15         @ public void accept(ResourceVisitor visitor) {
16             visitor.visit(cssResource: this);
17         }
18     }
```

Рис. 1.2 – Код класу CssResource

По аналогії до цього класу реалізовані наступні класи HtmlResource, JsResource, ImageResource – конкретні класи ресурсів із специфічними даними.

```
3  fx interface ResourceVisitor { 4 usages 1 implementation
4  fx      void visit(HtmlResource htmlResource); 1 usage 1 implementation
5  fx      void visit(CssResource cssResource); 1 usage 1 implementation
6  fx      void visit(JsResource jsResource); 1 usage 1 implementation
7  fx      void visit(ImageResource imageResource); no usages 1 implementation
8      }
```

Рис. 1.3 – Код класу ResourceVisitor

ResourceVisitor: Інтерфейс відвідувача з методами для кожного типу ресурсу.

```

3      class ResourceProcessor implements ResourceVisitor { no usages
4          @Override 1 usage
5  🚧 @
6          public void visit(HtmlResource htmlResource) {
7              System.out.println("Processing HTML content: " + htmlResource.getHtmlContent());
8          }
9
10         @Override 1 usage
11  🚧 @
12         public void visit(CssResource cssResource) {
13             System.out.println("Processing CSS content: " + cssResource.getCssContent());
14         }
15
16         @Override 1 usage
17  🚧 @
18         public void visit(JsResource jsResource) {
19             System.out.println("Processing JavaScript content: " + jsResource.getJsContent());
20         }
21
22         @Override 1 usage
23  🚧 @
24         public void visit(ImageResource imageResource) {
25             System.out.println("Processing Image: " + imageResource.getImageUrl());
26         }
27     }

```

Рис. 1.4 – Код класу ResourceProcessor

ResourceProcessor: Конкретний відвідувач, який обробляє ресурси.

```

~
3  ▶ public class Main {
4  ▶      public static void main(String[] args) {
5          BrowserResource html = new HtmlResource( htmlContent: "<html><body>Hello, World!</body></html>");
6          BrowserResource css = new CssResource( cssContent: "body { background-color: lightblue; }");
7          BrowserResource js = new JsResource( jsContent: "console.log('Hello, World!');");
8          BrowserResource image = new ImageResource( imageUrl: "https://example.com/image.png");
9
10         ResourceProcessor processor = new ResourceProcessor();
11
12         html.accept(processor);
13         css.accept(processor);
14         js.accept(processor);
15         image.accept(processor);
16     }
17 }

```

Рис. 1.5 – Код класу Main

Main: Демонстрація роботи патерну Visitor.

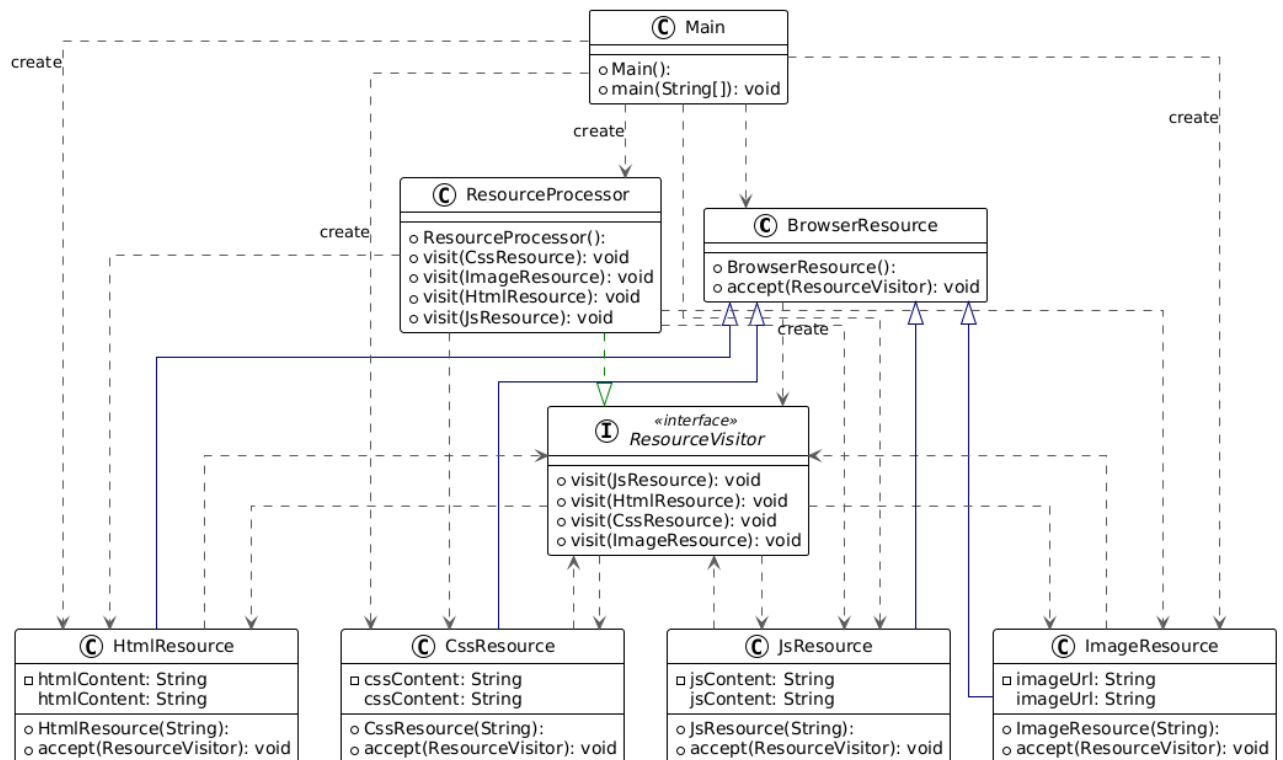


Рис. 1.6 – Структура шаблону Visitor

Діаграма класів демонструє реалізацію патерну Visitor, який використовується для розділення алгоритмів обробки від структури об'єктів. Патерн дозволяє додавати нові операції для існуючих класів без їх модифікації, що відповідає принципу відкритості/закритості.

Основна структура включає:

1. BrowserResource є абстрактним базовим класом для всіх ресурсів. Він визначає метод `accept(ResourceVisitor)`, який дозволяє ресурсам приймати відвідувачів.
2. Конкретні ресурси:

- HtmlResource зберігає HTML-контент
- CssResource містить CSS-стилі
- JsResource зберігає JavaScript-код
- ImageResource працює з URL-адресами зображень

Кожен з цих класів реалізує метод `accept()`, який викликає відповідний метод `visit()` відвідувача.

3. Інтерфейс ResourceVisitor визначає набір методів `visit()` для кожного типу ресурсу:

- `visit(HtmlResource)`
- `visit(CssResource)`

- visit(JsResource)
- visit(ImageResource)

4. ResourceProcessor є конкретною реалізацією відвідувача, що містить специфічну логіку обробки для кожного типу ресурсу.

5. Клас Main виступає як клієнт, який створює об'єкти та ініціює процес відвідування.

Патерн Visitor особливо корисний у цьому контексті, оскільки:

- Дозволяє додавати нові операції над ресурсами без зміни їх класів
- Розділяє логіку обробки різних типів ресурсів
- Забезпечує централізоване місце для обробки кожного типу ресурсу
- Спрощує додавання нових типів операцій над існуючими ресурсами

Загальна структура забезпечує гнучкість у роботі з різними веб-ресурсами, дозволяє легко розширювати функціональність системи та додавати нові алгоритми обробки без модифікації існуючих класів ресурсів.

Переваги використання патерна Visitor:

1. Розширюваність поведінки:

- Легко додавати нові види обробки ресурсів, створюючи нових відвідувачів, без змін у класах BrowserResource або їх підкласах.

2. Відокремлення логіки обробки:

- Обробка ресурсів зосереджена у класі-відвідувачі (ResourceProcessor), що спрощує підтримку та тестування.

3. Чітка структура коду:

- Кожен тип ресурсу (HtmlResource, CssResource, JsResource, ImageResource) фокусується на своїх даних, а логіка обробки передана відвідувачу.

4. Полегшення роботи з різними типами ресурсів:

- Патерн забезпечує централізовану обробку різнорідних об'єктів (HTML, CSS, JS, зображення), дозволяючи масштабувати рішення.

5. Полегшення тестування:

- Відвідувачі ізольовані від класів ресурсів, що дозволяє легко писати юніт-тести для кожного методу visit.

Недоліки використання патерна Visitor:

1. Складність додавання нових типів ресурсів:

- Якщо потрібно додати новий тип ресурсу (наприклад, VideoResource), доведеться оновлювати інтерфейс ResourceVisitor і всі його реалізації. Це може призвести до великого обсягу змін у коді.

2. Збільшення кількості класів:

- Кожен новий ресурс або поведінка вимагають створення нових класів, що може ускладнити структуру проекту.

3. Жорсткий зв'язок між ресурсами і відвідувачами:

- Типи ресурсів і методи відвідувачів пов'язані між собою, що може ускладнити внесення змін у майбутньому.

4. Непотрібний оверхед для простих систем:

- У невеликих або простих проєктах використання цього патерна може бути надмірним, особливо якщо типів ресурсів та поведінок небагато.

5. Порушення принципу відкритості/закритості (OCP):

- Додавання нових типів ресурсів вимагає змін у всіх реалізаціях інтерфейсу ResourceVisitor, що суперечить принципу OCP.

Можемо зробити висновок, що патерн **Visitor** ефективно справляється з розділенням логіки обробки різних типів ресурсів у веб-браузері. Він підходить для ситуацій, коли частіше додаються нові способи обробки, ніж нові типи ресурсів. Якщо ж кількість ресурсів розширюється частіше, варто розглянути інші підходи або ретельно зважити витрати на підтримку коду.

Реалізація: https://github.com/Elmir2/trpz_labs

Висновок: Під час виконання даної лабораторної роботи було досліджено структуру, призначення, основні властивості, переваги та недоліки шаблонів проєктування «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR». Наведені шаблони мають власні особливі переваги та недоліки, а також використовуються для досягнення різних цілей. Для реалізації частини майбутньої системи було обрано шаблон Visitor, який найкраще підходить у даному випадку. Далі було реалізовано обраний шаблон проєктування, розглянуто його призначення, структуру та досліджено його переваги та недоліки у використанні.